

Operating Systems

202.1.3031

Spring 2026 Assignment 3

Memory Management



Ben-Gurion University
of the Negev

Contents

1	Introduction	2
2	Submission Instructions	3
3	Background	4
3.1	The display device and framebuffer	4
3.2	The display daemon	4
4	Task 1: Memory-Mapped Framebuffer	5
5	Task 2: Zero-Copy Page Flip	7
5.1	Double Buffering	8
6	Task 3: Run Conway's Game of Life	10

1 Introduction

Welcome to the kernel. A place where virtual meets physical, where pages are currency, and where a single bad pointer can crash the machine.

In this assignment we will extend xv6 with a **display subsystem** and implement two different ways for userspace processes to draw pixels: memory-mapped I/O and zero-copy page flipping. Along the way you will get hands-on experience with the xv6 virtual memory system, page-table manipulation, device drivers, and the kernel-user interface.

This assignment is built on a **customised xv6 template** — not the generic xv6-riscv you used in Assignments 1 and 2. The template ships with a working VirtIO display driver, a kernel daemon, and stubs for the system calls you must implement.

Download it here:

<https://github.com/BGU-CS-OS/os262-assignment3-dist>

Good luck and have fun!

2 Submission Instructions

- Make sure your code compiles without errors and warnings and runs properly!
- We recommend that you comment your code and explain your choices, if needed. This would also be helpful for the discussion with the graders.
- Submit a **single copy** of xv6 containing all of the changes you made for all tasks, in a single `.tar.gz` or `.zip` file. All tasks must be completed in the same copy of xv6 without breaking the operating system or the other tasks.
- We advise you to work with git and commit your changes regularly. This will make it easier for you to track your progress, collaborate and maintain a working version of the code to compare to when things go wrong.
- Theoretical questions in this assignment are **not for submission**, but you are advised to understand the answers to these questions for the grading sessions.
- Submission is allowed **only in pairs** and **only via Moodle**. Email submissions will not be accepted.
- Before submitting, run the following command in your xv6 directory:

```
make clean
```

This will remove all compiled files as well as the `obj` directory.

- Help with the assignment and git will be given during office hours. Please email us in advance.

3 Background

Upon running xv6 with the provided template, you will find a new window showing the contents of the display. At boot, you should see “Hello World” displayed.

Demo programs are available to test each task:

```
$ show_map                # Task 1: memory-mapped (type 'exit' to quit)
$ show_flip Hello World   # Task 2: zero-copy flip
```

3.1 The display device and framebuffer

QEMU, the emulator that runs xv6 for us, is also capable of emulating a simple display device. The driver in `kernel/virtio_gpu.c` initializes it during the boot sequence and owns a **framebuffer** — 300 physical pages (set by `GPU_FB_PAGES`) holding 640×480 32-bit pixels in the array `fb[]`. The driver already exposes everything the kernel needs to use this device:

```
void virtio_gpu_init(void); // Initializes display device
void virtio_gpu_commit(void); // Displays buffer on screen (flush)
```

Internally, the device reads pixels from a **backing list** of physical pages. Static helper functions in `virtio_gpu.c` detach the current backing list and attach a new one. It is necessary to read and understand how these work for Task 2. Your tasks add the user-facing system calls on top of this driver. How you wire those system calls into the driver is up to you to decide.

3.2 The display daemon

A kernel daemon, `display_daemon`, calls `virtio_gpu_commit()` about every 100 ms (~ 10 fps), flushing the framebuffer to the display and triggering the presentation of the data within the framebuffer. Once you write pixels into the kernel framebuffer (directly or through a mapped user pointer), they

appear on screen within one tick **without any intervention required by userspace**.

4 Task 1: Memory-Mapped Framebuffer

In this task you will allow a user process to write pixels *directly* into the framebuffer without any system calls. You will need to map the kernel's physical framebuffer pages into the user process' virtual address space with read/write permissions (PTE_U | PTE_R | PTE_W). Once mapped, the user program can treat the returned pointer like a normal array, and the display daemon will periodically push the contents to the display.

To give a user process access to the framebuffer, we'll need to make the framebuffer pages reachable from the process' virtual address space: Start by installing all GPU_FB_PAGES pages of the kernel array fb[] into a process' page table at a chosen virtual address with user read/write permission, and provide a matching way to take that mapping back down when it is no longer needed.

Implement the `sys_map_display` syscall in `kernel/sysproc.c`, which will map the framebuffer into the calling process' address space and return the virtual address of the mapping. The syscall should accept one argument: the user-supplied virtual address `addr` with the following semantics:

- If `addr == 0`, the kernel **auto-selects** a suitable page-aligned VA (above `p->sz` and large enough to hold the framebuffer), installs the mapping there, and returns that VA.
- If `addr != 0`, it must be page-aligned and the resulting region [`addr`, `addr + GPU_FB_PAGES * PGSIZE`) must not collide with any existing mapping. Install the mapping there and return `addr`.
- Return -1 on any failure (bad alignment, collision, out of memory, etc.).

How exactly to install/remove the mapping (helper functions, where they live, how they cooperate with `kernel/vm.c`) is up to you. Read `kernel/vm.c` to see what xv6 already provides.

Once implemented, the provided user program `show_map` should work and the text you type should appear in the display window:

```
$ show_map
show_map: type text and press Enter to display it.
        Type 'exit' to clear the screen and quit.
> Hello World
```

In order to see your changes in action, you will need to run `xv6` with the `qemu-web` target, which exposes the display via a web interface and allows you to interact with it using your mouse and keyboard:

```
$ make qemu-web
```

This will start `xv6` with the web display. Open your browser and navigate to http://localhost:6080/vnc_auto.html to see the display window.

Task 1 — Memory-Mapped Framebuffer

1. Implement a way to map the kernel framebuffer pages into a user process' address space.
2. Implement the `sys_map_display` system call.
3. Keep it safe: remember that the framebuffer pages are kernel-owned, so think carefully about what should (and should not) happen to them when a mapping is taken down.
4. Test with `show_map`: text you type should appear in the display window.

For submission: Modified kernel files `kernel/virtio_gpu.c` and `kernel/sysproc.c` and any other files you modified.

5 Task 2: Zero-Copy Page Flip

In Task 1 the kernel framebuffer was shared with the user process: the device always reads from the same physical pages that the user writes into. This means that if the user is in the middle of drawing a frame while the daemon commits the display, the viewer may see a *partially-rendered* frame (also called *tearing*).

A classic solution to this problem is **double buffering**: the user draws into a *back buffer* and, when ready, atomically swaps it with the *front buffer* that the display device is reading.

In this task you will implement a **zero-copy page flip**: instead of copying the pixel data, the kernel simply *re-points* the device's backing list to the *physical pages* of the current process' back buffer. No pixel data crosses the kernel-user boundary; only the pointers in the backing list are updated.

The display device knows about a *backing list*: a list of (`physical_address`, `length`) entries that it reads from when `virtio_gpu_commit()` is called.

Two low-level helper functions in `kernel/virtio_gpu.c` manage this list. One sends a `RESOURCE_DETACH_BACKING` command, removing the current backing; the other sends `RESOURCE_ATTACH_BACKING` with an explicit array of `struct virtio_gpu_mem_entry` entries. Understanding what these functions do and how to call them correctly is part of this task.

The system calls `sys_flip_display` and `sys_map_display` are siblings: they both give the user process access to the framebuffer, but in different ways. They are provided in the template as stubs for you to implement.

Replace the `sys_flip_display` implementation with a working implementation that:

1. Reads the `void *buf` argument from userspace.
2. Validates that `buf` is page-aligned and that all `GPU_FB_PAGES` pages are mapped with user permission.
3. Calls `virtio_gpu_flip` to re-point the device to the current process' buffer.
4. Returns 0 on success, or -1 on error.

5.1 Double Buffering

With `flip_display` available, user programs can implement flicker-free animation using two framebuffers. `flip_display` re-points the display device at user pages, so the buffers **must be page-aligned**. `malloc` does not guarantee page alignment. Instead, use `sbrk` directly, which returns a page-aligned address as long as `p->sz` is page-aligned. This holds right after `exec`, i.e., when the process is initialized. The example below allocates two buffers, fills one with red and the other with blue, and alternates the display between them so you can see the swap happen:

```
uint32 *buf[2];
buf[0] = (uint32 *)sbrk(FB_BYTES);
buf[1] = (uint32 *)sbrk(FB_BYTES);
for (int i = 0; i < FB_BYTES / 4; i++) {
    buf[0][i] = 0x00FF0000;    // Red (order is Blue Green Red X)
    buf[1][i] = 0x000000FF;    // Blue
}
for (int i = 0; i < 50; i++) {
    flip_display(buf[i & 1]); // display now reads buf[i & 1]
    sleep(1);
}
```

After `flip_display(buf[i & 1])`, the device reads from that buffer's physical pages. To create an animation, you would draw the *next* frame into the back buffer (the one the device is not currently reading) and then flip. This way the device never sees a half-drawn image.

Once implemented, `show_flip` should work:

```
$ show_flip Hello World
```

The text should appear in the display window immediately.

Task 2 — Zero-Copy Page Flip

1. Implement `virtio_gpu_flip` in `kernel/virtio_gpu.c`.
2. Implement `sys_flip_display` in `kernel/sysproc.c`.
3. Test with `show_flip`: text should appear on the GPU display.

For submission: Modified `kernel/virtio_gpu.c` and `kernel/sysproc.c`.

6 Task 3: Run Conway's Game of Life

You that now have both GPU mechanisms, it's time to put them to the test with a real animation. The user program `gol.c` is a working implementation of **Conway's Game of Life** that animates an 80×60 grid on the 640×480 display for 100 generations. It exercises both interfaces you built:

```
$ gol           # default: flip mode
$ gol flip      # uses flip_display (double-buffered)
$ gol map       # uses map_display (writes into the kernel fb)
```

Read `user/gol.c` as a reference for how a real user program drives the display. If Tasks 1 and 2 are correct, both modes should animate smoothly and exit cleanly.

Task 3 — Run the Game of Life

Run `gol flip` and `gol map`. Both should display the animation for 100 steps and exit cleanly.

For Submission: Nothing new for this task.

Questions — not for submission

1. What is the difference between `map_display` and `flip_display` from the kernel's perspective? In each case, describe what physical pages the display device is backed by after the call.
2. In `flip_display`, why must the kernel walk the page table page-by-page rather than simply passing the user virtual address `fbva` to the device?
3. Why is double-buffering important in flip mode? What visual artifact would you observe if you drew into the same buffer that the device is currently reading?
4. In map mode (`show_map`, `gol map`), the user writes pixels through a pointer and relies on the display daemon to flush the display. What would happen if the daemon were not running? How would you let user code force a flush by itself if it needed to?
5. A process calls `flip_display` and then exits. The device now points at the physical pages that were allocated for the user process' framebuffer. What happens to those pages after the process exits? What should the kernel do to avoid this problem?
6. Which one uses more memory bandwidth: `flip_display` or a fake `display_write` system call that copies the entire framebuffer from user space to kernel space? For a $640 \times 480 \times 4$ byte buffer, how many bytes are moved in each case?
7. In flip mode, can the user safely read from `buf[i & 1]` (the back buffer) while the daemon is committing? Is there a race condition?